

TP Chapitre 0 : Compilation et Débogage

Pistes et Indices

Dr. El Hadji Bassirou TOURÉ
Département de Mathématiques et Informatique
Université Cheikh Anta Diop de Dakar

À propos de ce document

Ce document fournit des indices pour les exercices du TP. Il ne contient pas de solutions complètes.

Exercice 1 : Phases et optimisations

Pistes – Partie A (fichiers intermédiaires)

- Le préprocesseur effectue des **substitutions textuelles**. Après cette phase, les macros n'existent plus dans le code.
- Utilisez `grep "TAILLE" phases.i` pour vérifier si la macro apparaît.
- La commande `nm` affiche la table des symboles :
 - T : symbole défini dans la section texte (code exécutable)
 - D : symbole défini dans la section data (données initialisées)
 - U : symbole non défini, doit être résolu au linkage
- Comparez `nm phases.o` et `nm phases` : les U disparaissent après linkage.

Pistes – Partie B (optimisations)

- L'optimisation `-O3` peut **inliner** les fonctions (remplacer l'appel par le code).
- Le compilateur peut précalculer `somme_carres(100)` à la compilation si la valeur est constante.
- Cherchez `call somme_carres` dans l'assembleur `-O0` vs `-O3`.
- L'inlining peut **augmenter** la taille du code assembleur (duplication).

Exercice 2 : Multi-fichiers et erreurs

Pistes – Compilation séparée

- Lors de la compilation de `main.c`, le compilateur ne connaît que les **déclarations** (via le header).
- Les symboles U dans `main.o` seront résolus par le linker.
- Le header ne génère PAS de code, il ne fait que déclarer.

Pistes – Diagnostic d'erreurs

- **Erreur de compilation** : syntaxe incorrecte. Message avec numéro de ligne.
- **Erreur de liaison** : symbole manquant. Message `undefined reference`.
- **Définition multiple** : même symbole défini dans deux `.o`. Message `multiple definition`.
- Une déclaration `extern` sans définition compile mais ne linke pas.

Exercice 3 : GDB

Pistes – Bugs à trouver

- **Bug 1** : `max = 0` ne fonctionne pas pour des valeurs négatives. Initialiser avec `tab[0]` ou `INT_MIN`.
- **Bug 2** : `i <= taille` fait un tour de trop. Dernier indice valide = `taille - 1`.
- **Bug 3** : `somme / taille` est une division entière. Utiliser `(double)somme / taille`.

Pistes – Commandes GDB utiles

- `watch variable` : arrêt à chaque modification de la variable.
- `print tab[5]` : affiche la valeur hors limites (comportement indéfini).
- `backtrace` : affiche la pile d'appels.
- `next` : exécute sans entrer dans les fonctions. `step` : entre dans les fonctions.

Exercice 4 : Valgrind

Pistes – Types d'erreurs

- **Invalid read of size 4** : lecture hors des limites allouées.
- **Conditional jump depends on uninitialised value** : utilisation de variable non initialisée.
- **Definitely lost: N bytes** : mémoire allouée jamais libérée.
- Fuite attendue : `100 * sizeof(int) = 400 bytes`.

Pistes – Corrections

- Accès hors limites : changer `i <= taille` en `i < taille`.
- Variable non initialisée : `int resultat = 0;` ou ajouter un `else`.
- Fuite mémoire : ajouter `free(autre);` avant `return`.
- Objectif final : `ERROR SUMMARY: 0 errors et no leaks are possible`.

Makefile suggéré

Structure

```
1 CC = gcc
2 CFLAGS = -Wall -Wextra -g -O0
3
4 TARGETS = phases buggy memoire programme
5
6 all: $(TARGETS)
7
8 phases: phases.c
9     $(CC) $(CFLAGS) -o $@ $<
10
11 programme: math_utils.o main.o
12     $(CC) -o $@ $^
13
14 %.o: %.c
15     $(CC) $(CFLAGS) -c $< -o $@
16
17 clean:
18     rm -f $(TARGETS) *.o *.i *.s
19
20 .PHONY: all clean
```